Completion Frontiers and Quotient Geometry in Deterministic Pushdown Automata for Rational Binary Counting

Alp Eren Bütün
Independent Researcher
GitHub: github.com/alperenbutun

August 2026

Abstract

We study completion-aware deterministic pushdown representations of

$$L_{p,q} = \{w \in \{0,1\}^* : qN_1(w) = pN_0(w)\}, \qquad \gcd(p,q) = 1,$$

beginning with a concrete DPDA for $L_{2,1}$. Beyond recognition, the motivating machine supports an end-of-input operation that converts its current stack into a canonical shortest continuation. We formalize this operation with a right endmarker, prove an exact state-dependent reachable-store normal form, derive a state-potential invariant, and show that the symbol-presence query used by the design is determined by the first two stack positions; depth two is necessary.

For general p, q, the residual

$$\rho_{p,q}(w) = qN_1(w) - pN_0(w)$$

exactly indexes prefix left quotients. Shortest completions are directed geodesics in the quotient graph

$$r \xrightarrow{1} r + q, \qquad r \xrightarrow{0} r - p.$$

Its fundamental balanced cycle has length $p + q$. From this geometry we obtain a canonical two-block shortest completion, exact visibility depth

$$d_{p,q} = \max(p, q),$$

and an exact finite completion frontier

$$K_{p,q} = p + q - 1.$$

The same almost-cycle word witnesses both extremal bounds. We then factor the bounded completion frontier into finite control without losing the residual, prefix quotient, or canonical shortest continuation. A compact four-symbol realization is given as a secondary descriptional consequence. The contribution is the structural chain from a completion-oriented stack design to exact bounded visibility, completion-frontier factorization, quotient semantics, and shortest-continuation decoding.

1 Introduction

The language

$$L_{2,1} = \{w \in \{0,1\}^* : N_1(w) = 2N_0(w)\}$$

is easy to recognize with counter-like memory: one may track the signed difference $N_1 - 2N_0$. Deterministic one-counter models and signed letter-count differences are classical [1, 3, 4]. Recognition alone, however, does not explain the internal organization of the motivating pushdown design studied here.

That design stores something more operational: after a prefix has been read, its stack encodes *what is still needed* to reach the language. When the binary input ends, a completion operation converts the current store into a canonical shortest continuation. The central problem of this paper is therefore not whether $L_{2,1}$ is deterministic context-free, but how this completion-oriented store works, why a bounded inspection of it suffices, and what structure survives for the full family $L_{p,q}$.

This viewpoint leads to three concrete questions.

(i) What stack contents are actually reachable in the original DPDA, and why does inspecting only a bounded prefix of the stack suffice for the global symbol-presence query used by the design?

(ii) Does the completion mechanism admit an exact generalization to

$$L_{p,q} = \{w : qN_1(w) = pN_0(w)\}$$

for arbitrary coprime p, q?

(iii) Can the bounded completion information be separated from the unbounded unary magnitude, so that the stack representation, finite control, prefix quotient, and shortest continuation become different views of one underlying object?

The answer to all three is yes. For the original 2:1 machine we prove an exact state-dependent reachable-store normal form and a potential invariant. Its end-of-input stack normalizer produces a canonical shortest completion, and the set of symbols occurring anywhere in the normalized stack is determined by the first two positions; the depth-two bound is tight.

For the general family, define the residual

$$\rho_{p,q}(w) = qN_1(w) - pN_0(w).$$

A continuation with x zeros and y ones completes w exactly when

$$px - qy = \rho_{p,q}(w).$$

The residual is therefore both the arithmetic obstruction to acceptance and the complete prefix-left-quotient invariant. It induces the directed quotient graph

$$r \xrightarrow{1} r + q, \qquad r \xrightarrow{0} r - p.$$

The shortest-continuation problem is exactly the directed shortest-path problem from the current residual to zero. The graph has a fundamental directed cycle of length $p + q$;

deleting its Parikh vector (q, p) gives a complete geodesic reduction. This geometry yields two exact memory parameters:

$$\boxed{d_{p,q} = \max(p, q)} \qquad \text{and} \qquad \boxed{K_{p,q} = p + q - 1}.$$

The first is the least prefix depth needed to recover symbol support from the canonical shortest-completion representation; the second is the maximum finite boundary correction in its residue-unary factorization. The same extremal shortest word realizes both bounds.

The direct factorization places this finite boundary in control and leaves only a unary magnitude in the stack. The resulting normalized configuration determines the exact residual, hence the exact prefix quotient, exact distance to acceptance, and a canonical shortest continuation. A further paired-sign encoding shows how a small increase of the stack alphabet can merge positive and negative residue roles, giving compact four-symbol DPDAs with $\max(p, q) + 1$ states without an endmarker and $\max(p, q)$ states with one. These construction bounds are secondary: counting and descriptional complexity for pushdown/counter models, residue-in-control normalization, and state/stack-symbol tradeoffs have substantial prior art [10, 3, 8, 9, 13]. Our emphasis is the exact completion structure that explains why these representations arise from the original stack design.

Contributions. The paper is organized around six results rather than a long list of isolated lemmas: (1) an exact structural theorem for the motivating completion-aware DPDA; (2) a quotient-geodesic theorem for $L_{p,q}$; (3) tight visibility and frontier bounds; (4) a lossless completion-frontier factorization; (5) an explicit compact DPDA realization; and (6) a configuration semantics that recovers the quotient, exact distance to acceptance, and a canonical shortest witness. The first four results form the conceptual core; the compact realization is a derived descriptional consequence.

Scope of novelty. We do not claim that deterministic counting, one-counter recognition, prefix completion as a general task, shortest paths in one-counter systems, regular geodesic normal forms, store-language analysis, or bounded stack-prefix-to-state conversions are new [5, 6, 7, 13]. The novelty claim is deliberately narrower: the exact theorem chain extracted from the supplied completion-aware DPDA and generalized to the full rational binary counting family.

2 The original 2023 completion-aware DPDA

The motivating 2:1 machine was developed by the author in 2023, before the present generalization and proof analysis. We therefore refer to it as the *original 2023 design*, rather than as a classical or previously published "historical" automaton. The original implementation material is preserved through the author's GitHub profile, `https://github.com/alperenbutun`. The purpose of this section is to retain that design as the starting object while separating three roles that are easy to conflate: ordinary recognition, acceptance of the empty word, and end-of-input completion.

2.1 Two formal variants

The two automata in figures 1 and 2 implement the same center/left/right stack logic. They differ only in whether the empty input is accepted. The notation

$$a, A/\gamma$$

means: read input symbol a, replace stack top A by γ, and move along the indicated arc. The symbol Z_0 is the bottom marker and ε denotes the empty word.

The completion rules are displayed in dashed callout boxes rather than as ordinary graph edges. The dashed shape, not color alone, distinguishes them in grayscale. They are enabled only when the right endmarker $\dashv$ is the next unread symbol. Formally, each such rule can be directed to a dedicated completion sink q_{comp}, omitted from the drawings for clarity. This point is essential: replacing the original completion rules by ordinary ε-moves would conflict with input-consuming transitions from the same state/top pair and would violate the standard DPDA determinism condition. With $\dashv$, no such conflict occurs.

How to read the diagrams. The center state C carries the ordinary cancellation logic. The side states L and R are transient mismatch modes: they do not represent a second independent counter, but a finite correction to the meaning of the stack. Solid arrows process 0 and 1; the dashed blue boxes list only the rules that become available after the binary input has ended. The two figures have identical center/left/right behavior. Their only intended difference is whether the empty input is accepted.

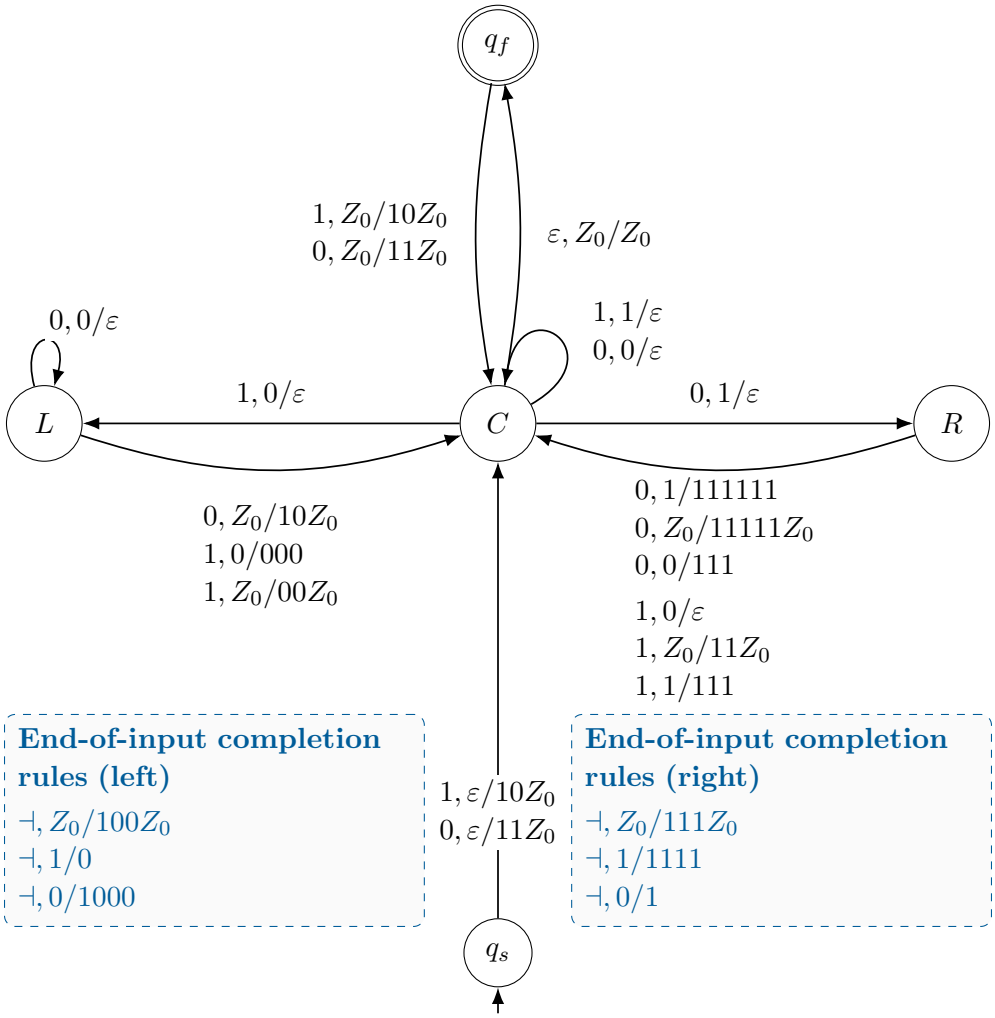

Figure 1: Formalized redrawing of the original 2:1 DPDA variant with the empty input excluded. Solid arrows are ordinary recognition transitions. Dashed callout boxes list the completion rules enabled only by the right endmarker $\dashv$ after the binary input has been exhausted; the same distinction remains visible in grayscale. The left-state rule $\dashv, 1/0$ is retained for fidelity to the original completion list, but its source condition is unreachable because every reachable left-state stack belongs to 0^*. The source design is archived at `https://github.com/alperenbutun/dpda-ones-double-zeros`.

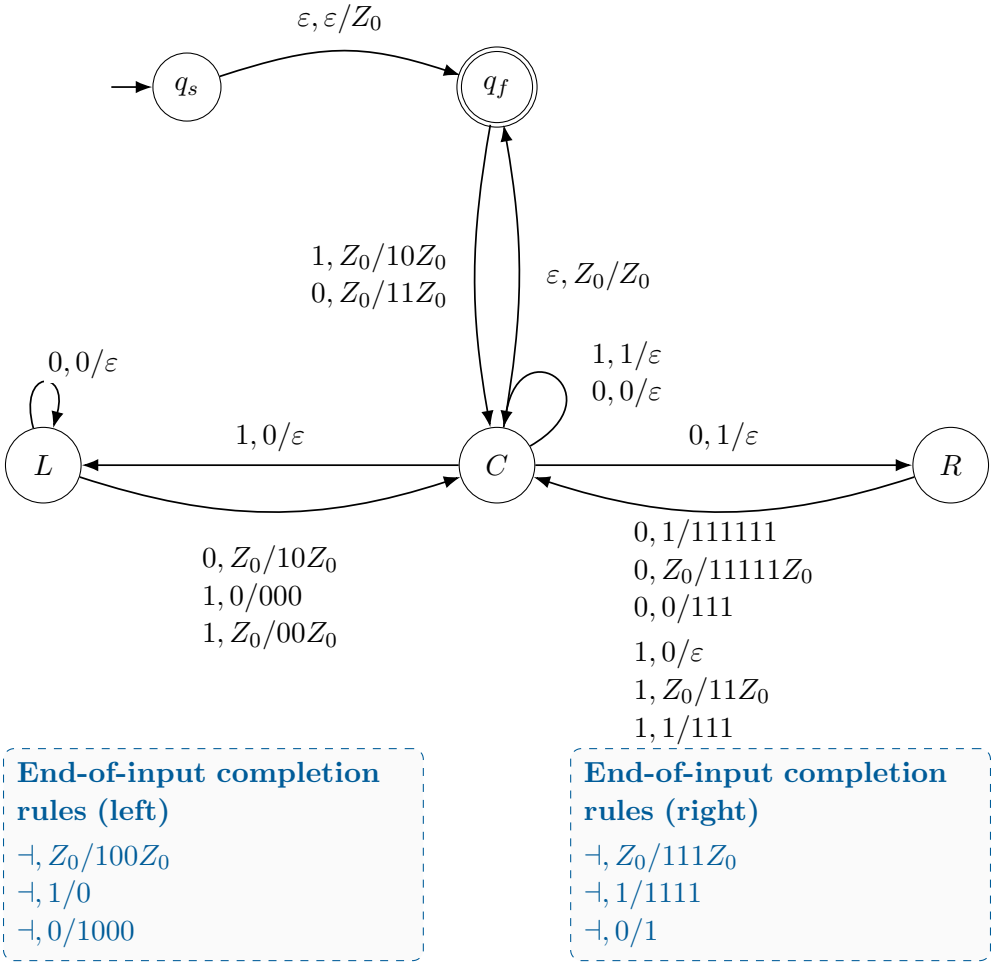

Figure 2: Formalized redrawing of the original 2:1 DPDA variant with the empty input included. The center/left/right recognition logic and the dashed end-of-input completion callouts are the same as in figure 1; the initialization/final-state structure additionally accepts ε. Hence this variant recognizes the standard language $L_{2,1}$, while the previous variant recognizes $L_{2,1} \setminus \{\varepsilon\}$. The source design is archived at `https://github.com/alperenbutun/dpda-ones-twice-zeros`.

The states q_s and q_f in the drawings are an initialization/acceptance wrapper around the semantic core $\{C, L, R\}$. To state the store theorems without conflating those roles, write a physical stack as SZ_0, where $S \in \{0,1\}^*$ is the *logical stack*, and define the core projection

$$\pi(C, SZ_0) = (C, S), \qquad \pi(L, SZ_0) = (L, S), \qquad \pi(R, SZ_0) = (R, S),$$

together with the wrapper conventions

$$\boxed{\pi(q_s, \varepsilon) = (C, \varepsilon), \qquad \pi(q_f, Z_0) = (C, \varepsilon).}$$

The source state q_s occurs only during initialization and is not a semantic core state; its projection is used only to assign the empty prefix its zero-residual core meaning. In the empty-input-included variant, the initial wrapper move reaches (q_f, Z_0), which therefore projects to the empty center configuration. Whenever a balanced prefix places the physical machine in (q_f, Z_0), the same projection identifies it with the zero-residual center configuration. In what follows, "the configuration after a processed prefix" means this core-projected configuration unless the wrapper is mentioned explicitly.

Let C, L, R denote the center, left, and right core states. We use the leftmost logical stack symbol as the top. Ignoring the endmarker completion callouts, the solid recognition transitions induce the following reachable logical stores.

Theorem 2.1 (Original-machine reachable-store normal form)**.** *For every binary prefix processed by either formal variant, if (q, S) is its core-projected configuration, then $S \in \mathcal{S}_q$, where*

$$\mathcal{S}_C = 0^* \cup 1^* \cup \{10\}, \qquad \mathcal{S}_L = 0^*, \qquad \mathcal{S}_R = 1^* \cup \{0\}.$$

Proof. The proof is by induction on consumed input length. The empty logical stack is in the center family. Each solid transition preserves one of the displayed forms: center loops either cancel a matching top symbol or create a short homogeneous block; a center-to-left mismatch produces only a zero stack in L; a center-to-right mismatch produces either a one stack or the exceptional one-symbol stack 0 in R; and every side-to-center transition returns to 0^*, 1^*, or the exceptional center word 10. No transition enters L with top 1, which also proves the reachability note in the figure caption. □

The store normal form is strengthened by a numerical invariant. For an original-machine logical stack word S, set

$$D(S) = 2N_0(S) - N_1(S),$$

and define the state potential

$$\Phi(C) = 0, \qquad \Phi(L) = 3, \qquad \Phi(R) = -3.$$

Theorem 2.2 (Original-machine residual invariant)**.** *After every consumed prefix w, let (q, S) be its core-projected configuration. Then*

$$\boxed{\rho_{2,1}(w) = D(S) + \Phi(q),}$$

where $q \in \{C, L, R\}$ and S is the logical stack.

Proof. The projected empty-prefix configuration is (C, ε), where both sides are zero. Compare the change in $\rho_{2,1}$ with the change in $D + \Phi$ for each solid core transition; the wrapper projections preserve the zero-residual center meaning. For example, the left-state cancellation $0, 0/\varepsilon$ changes both quantities by -2. The remaining transitions are finite in number and satisfy the same equality. □

This gives a precise meaning to the original cancellation intuition: local stack cancellation is an invariant-preserving arithmetic reduction rather than an ad hoc simplification.

2.2 Completion as a deterministic end-of-input operation

Write an instantaneous configuration as (q, u, S), with current state q, unread input u, and stack S. The one-step relation $\vdash_M$ is induced by the transition function. Thus the recognition phase of a completed input has the form

$$(q_0, w \dashv, Z_0) \vdash_M^* (q, \dashv, S).$$

At this point the next symbol is uniquely $\dashv$, so a side-state completion rule is deterministic even when the same state/top pair has ordinary transitions on 0 or 1.

Lemma 2.3 (Endmarker determinism)**.** *Replace each original post-input completion rule by the corresponding $\dashv$-labeled rule and direct it to a completion sink with no outgoing transitions. Then the resulting completion-extended machine is deterministic in the standard DPDA sense.*

Proof. For a fixed state/top pair, ordinary recognition moves are selected by input symbol 0 or 1, whereas completion is selected only by $\dashv$. Hence a configuration has at most one successor. The completion sink contributes no competing move. □

The completion rules are not needed for recognition, but they expose what the stack means after a prefix. At end of input, first apply the core projection above. If the projected state is L or R, apply its unique side-state completion rule; if it is C (including a physical (q_f, Z_0) configuration), no correction is needed.

Theorem 2.4 (Original-machine shortest-completion extraction)**.** *After end-of-input normalization, the original-machine stack is*

$$\widehat{S}(w) = \begin{cases} 0^m, & \rho_{2,1}(w) = 2m, \quad m \in \mathbb{N}_0, \\ 10^{m+1}, & \rho_{2,1}(w) = 2m+1, \quad m \in \mathbb{N}_0, \\ 1^m, & \rho_{2,1}(w) = -m, \quad m \geq 1. \end{cases}$$

Interpreted as a continuation word, $\widehat{S}(w)$ is a shortest completion of w to $L_{2,1}$.

Proof. By theorems 2.1 and 2.2, each side-state stack belongs to a one-parameter normal form and has exactly the potential offset corrected by its endmarker rule. After correction the center stack has potential zero, so its stack weight equals the residual. On the center family $0^* \cup 1^* \cup \{10\}$, the displayed residual-to-stack map is injective. For residual r, any continuation with x zeros and y ones must satisfy $2x - y = r$. The three displayed forms realize the unique minimum nonnegative Parikh pair, hence are shortest. □

2.3 Worked example: the prefix 111

A short run shows why the completion rules are useful. Use the empty-input-excluded presentation and ignore Z_0 when describing the logical stack. Starting in the center with an empty logical stack,

$$(C, 111\ \dashv, \varepsilon) \vdash_M (C, 11\ \dashv, 10) \vdash_M (C, 1\ \dashv, 0) \vdash_M (L, \dashv, \varepsilon).$$

The three consumed symbols give

$$\rho_{2,1}(111) = 3.$$

At this point no binary input remains. The left-state endmarker rule

$$\dashv, Z_0/100Z_0$$

therefore produces the completion word 100. Indeed,

$$111100 \in L_{2,1},$$

because the concatenation contains four ones and two zeros. Moreover, any completion with x zeros and y ones must satisfy

$$2x - y = 3.$$

Its unique minimum nonnegative solution is $(x, y) = (2, 1)$, so every shortest completion has length 3, and 100 is the canonical one selected by the machine.

This example also illustrates the role of the side state: the empty logical stack in L does *not* mean zero residual. The missing value is carried by the state potential $\Phi(L) = 3$, and the endmarker rule transfers that finite correction into the completion stack.

The original "search only the first two stack symbols" motivation can now be stated exactly.

For a binary word v, let

$$\mathrm{alph}(v) = \{a \in \{0, 1\} : a \text{ occurs in } v\}$$

be its symbol support, and let $\mathrm{pref}_d(v)$ denote the prefix of v of length $\min\{d, |v|\}$. In particular, $\mathrm{pref}_d(\varepsilon) = \varepsilon$.

Theorem 2.5 (Tight depth-two visibility)**.** *For every completion-normalized original-machine stack S,*

$$\mathrm{alph}(S) = \mathrm{alph}(\mathrm{pref}_2(S)).$$

Depth two is necessary.

Proof. The normalized family consists only of 0^*, 1^*, and 10^+. Any symbol occurring anywhere therefore occurs in the first two positions. Necessity follows from the normalized stacks 1 and 10: they have the same top symbol but different global symbol sets. □

2.4 Roadmap from the original machine to the general theory

The remainder of the paper progressively removes accidental details of the original drawing while preserving its completion semantics. The state potential first exposes the signed residual carried jointly by control and stack. That residual then becomes the exact prefix-quotient coordinate, so shortest completions can be studied as directed geodesics. Their two-block normal form yields the bounded visibility and finite-frontier theorems; the frontier can then be moved into finite control without losing the residual. Finally, the paired-sign encoding compresses the control representation while retaining exact quotient, distance, and shortest-witness decoding. Figure 3 summarizes this chain.

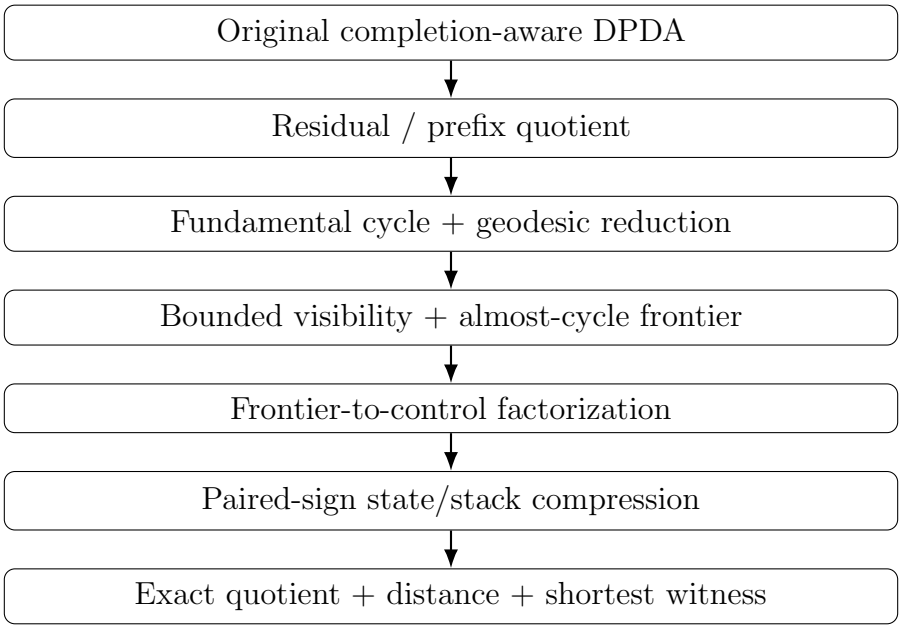

Figure 3: Proof architecture. The original machine is the starting object; the compact counter-like realization is a derived endpoint rather than the motivation.

3 Residuals, quotients, and shortest completions

Fix coprime $p, q \geq 1$ and write

$$\rho_{p,q}(w) = qN_1(w) - pN_0(w).$$

For a continuation z, let $x = N_0(z)$ and $y = N_1(z)$.

Lemma 3.1 (Completion equation)**.**

$$z \in w^{-1}L_{p,q} \iff px - qy = \rho_{p,q}(w).$$

Proof. Expand the equality $qN_1(wz) = pN_0(wz)$ and collect the counts contributed by z. □

All integer solutions of $px - qy = r$ differ by the kernel vector (q, p).

Proposition 3.2 (Affine completion ray)**.** *For every $r \in \mathbb{Z}$, there is a unique nonnegative solution (x_r, y_r) of $px - qy = r$ minimizing $x + y$. Every nonnegative solution is*

$$(x_r + qk, y_r + pk), \qquad k \in \mathbb{N}_0.$$

Consequently all completion lengths are

$$\ell_{\min}(r) + (p+q)k, \qquad k \in \mathbb{N}_0,$$

and the number of shortest completion words is

$$\binom{x_r + y_r}{x_r}.$$

Proof. Because $\gcd(p, q) = 1$, the Diophantine equation $px - qy = r$ has an integer solution (x_0, y_0). Its full integer solution set is

$$(x_0 + qk,\ y_0 + pk), \qquad k \in \mathbb{Z}.$$

For all sufficiently large k both coordinates are nonnegative. Let k_* be the least integer for which this happens and put

$$(x_r, y_r) = (x_0 + qk_*,\ y_0 + pk_*).$$

Then every other nonnegative solution has parameter $k_* + j$ with $j \in \mathbb{N}_0$, hence is exactly $(x_r + qj, y_r + pj)$. Since each positive j increases the length by $p + q$, the pair (x_r, y_r) is the unique minimum-length nonnegative solution. The length progression follows immediately. Finally, every shortest completion has the same Parikh pair (x_r, y_r), and there are $\binom{x_r+y_r}{x_r}$ orderings of that pair. □

The minimum pair has a particularly simple modular description.

Theorem 3.3 (Two-block shortest completion)**.** *For $r > 0$, let $y_r \in \{0, \ldots, p-1\}$ be the unique solution of*

$$qy_r \equiv -r \pmod{p},$$

and put

$$x_r = \frac{r + qy_r}{p}.$$

Then

$$C(r) = 1^{y_r} 0^{x_r}$$

is a shortest completion. For $r < 0$, let $x_r \in \{0, \ldots, q-1\}$ satisfy

$$px_r \equiv r \pmod{q},$$

put

$$y_r = \frac{px_r - r}{q},$$

and set

$$C(r) = 0^{x_r} 1^{y_r}.$$

Finally $C(0) = \varepsilon$.

Proof. The modular condition selects the unique solution whose bounded coordinate is in the indicated complete residue system. Any other integer solution adds or subtracts (q, p). Subtraction would make the bounded coordinate negative, so the selected pair is the unique minimum nonnegative pair. The displayed word is a fixed ordering of that pair. □

Example. For $p = 3$, $q = 2$, and residual $r = 4$, the congruence

$$2y \equiv -4 \pmod 3$$

gives $y = 1$, hence $x = (4 + 2)/3 = 2$. Therefore

$$C(4) = 100, \qquad \ell_{\min}(4) = 3.$$

All other completion Parikh pairs are

$$(2, 1) + k(2, 3),$$

so their lengths are $3 + 5k$. The increment $5 = p + q$ will reappear in section 4 as the length of the fundamental directed cycle.

Corollary 3.4 (Exact right-extension distance)**.** *For $r > 0$,*

$$\ell_{\min}(r) = \frac{r + (p + q)y_r}{p};$$

for $r < 0$,

$$\ell_{\min}(r) = \frac{(p + q)x_r - r}{q}.$$

Lemma 3.5 (Residual surjectivity)**.** *For every $r \in \mathbb{Z}$ there is a word $w \in \{0, 1\}^*$ with $\rho_{p,q}(w) = r$.*

Proof. Because $\gcd(p, q) = 1$, Bézout's identity gives integers x_0, y_0 satisfying $qy_0 - px_0 = r$. Every pair

$$(x_0 + qk,\ y_0 + pk), \qquad k \in \mathbb{Z},$$

has the same value of $qy - px$. For sufficiently large k, both coordinates are nonnegative. Any binary word with that many zeros and ones therefore has residual r. □

The residual is not merely sufficient for membership; it exactly classifies future behavior.

Theorem 3.6 (Prefix quotient classification)**.** *For all $u, v \in \{0, 1\}^*$,*

$$\boxed{u^{-1}L_{p,q} = v^{-1}L_{p,q} \iff \rho_{p,q}(u) = \rho_{p,q}(v),}$$

Proof. If the residuals agree, theorem 3.1 gives identical completion equations. Conversely, suppose the residuals are $r \neq s$. By theorem 3.2, choose a nonnegative pair (x, y) satisfying $px - qy = r$, and let z be any binary word with that Parikh pair. Then $z \in u^{-1}L_{p,q}$ by theorem 3.1, but the same word cannot satisfy the completion equation for residual s. Hence $z \notin v^{-1}L_{p,q}$ and the quotients differ. □

Remark 3.7. The displayed quotient theorem intentionally avoids a claim of new quotient theory: left quotients are classical. What matters later is that a normalized DPDA configuration realizes this quotient index exactly.

4 Quotient geometry and balanced-cycle reduction

Define the labeled directed graph $\mathcal{G}_{p,q}$ with vertex set $\mathbb{Z}$ and edges

$$r \xrightarrow{1} r+q, \qquad r \xrightarrow{0} r-p.$$

By theorems 3.5 and 3.6, every integer residual occurs and two prefixes have the same left quotient exactly when their residuals agree. Thus the vertices of $\mathcal{G}_{p,q}$ are precisely the prefix left quotients of $L_{p,q}$.

Theorem 4.1 (Directed completion metric)**.** *If $d_{\mathcal{G}}$ is directed shortest-path distance, then*

$$\boxed{d_{\mathcal{G}}(r,s) = \ell_{\min}(r-s).}$$

Proof. A path from r to s containing x zeros and y ones satisfies $s - r = qy - px$, equivalently $px - qy = r - s$. Minimizing path length $x + y$ is exactly the completion minimization problem. □

Theorem 4.2 (Fundamental directed cycle)**.** *Every nonempty directed closed walk has*

$$N_0 = qk, \qquad N_1 = pk$$

for some $k \geq 1$. Hence every directed cycle length is a positive multiple of $p+q$, and the shortest cycle length is

$$\boxed{g_{p,q} = p+q.}$$

Proof. A closed walk has zero net residual change, so $qN_1 = pN_0$. Coprimality gives the displayed form. Conversely any ordering of q zeros and p ones is a closed walk of length $p+q$. □

The original-machine cancellation operation now has a general global analogue.

Theorem 4.3 (Balanced-cycle reduction)**.** *Let a word have Parikh pair (x,y). Put*

$$k = \min\left(\left\lfloor \frac{x}{q} \right\rfloor, \left\lfloor \frac{y}{p} \right\rfloor\right).$$

Then

$$(x', y') = (x - kq, y - kp)$$

has the same residual and is the unique minimum Parikh pair for that residual.

Proof. Subtracting (q,p) preserves $px - qy$. Maximality of k gives $x' < q$ or $y' < p$. Since every solution differs by an integer multiple of (q,p), no further positive multiple can be removed while remaining nonnegative; hence the pair is minimum and unique. □

Corollary 4.4 (Exact geodesic domain)**.** *A word z is shortest among all words with its residual iff*

$$\boxed{N_0(z) < q \quad \textit{or} \quad N_1(z) < p.}$$

Thus

$$\mathrm{Geo}_{p,q} = \{z : N_0(z) < q\} \cup \{z : N_1(z) < p\}$$

is regular.

This family also admits exact quotient growth.

Proposition 4.5 (Exact quotient spheres)**.** *Let*

$$S_n = \{r \in \mathbb{Z} : d_{\mathcal{G}}(r, 0) = n\}.$$

Then

$$\boxed{|S_n| = \min(n + 1, p + q).}$$

Consequently the sphere generating function is

$$\sum_{n \geq 0} |S_n| z^n = \frac{1 - z^{p+q}}{(1 - z)^2}.$$

Proof. A length-n Parikh pair is $(x, n-x)$ and has residual $(p+q)x - qn$, which is injective in x. By theorem 4.4, it is geodesic iff $x < q$ or $n - x < p$. If $n < p + q$, all $n + 1$ pairs qualify. If $n \geq p + q$, exactly $p + q$ qualify. Summing the resulting sequence gives the rational generating function. □

Regular geodesic languages and shortest paths in one-counter systems are established themes [7, 5]; theorems 4.1 to 4.3 and 4.5 are used here as exact structural formulas for this particular two-generator quotient graph and as the bridge back to the stack completion mechanism.

5 Exact visibility and the completion frontier

The two-block selector $C(r)$ is more than a convenient shortest representative. Its bounded first block controls how much of the word must be inspected before all symbol types occurring in the completion are known.

Definition 5.1 (Visibility depth)**.** Let $d_{p,q}$ be the minimum d such that every canonical shortest completion satisfies

$$\mathrm{alph}(C(r)) = \mathrm{alph}(\mathrm{pref}_d(C(r))).$$

Theorem 5.2 (Exact bounded visibility)**.**

$$\boxed{d_{p,q} = \max(p, q).}$$

Proof. For $r > 0$, $C(r) = 1^y 0^x$ with $y < p$. If a zero occurs at all, it appears by position p. For $r < 0$, $C(r) = 0^x 1^y$ with $x < q$, so any one appears by position q. Hence $d_{p,q} \leq \max(p, q)$.

For tightness, if $p > q$, the residual q has

$$C(q) = 1^{p-1} 0^q,$$

so depth $p - 1$ sees only ones although zero occurs later. If $q > p$, the symmetric witness is

$$C(-p) = 0^{q-1} 1^p.$$

For $p = q = 1$, depth one is necessary and sufficient. □

The original-machine visibility bound in theorem 2.5 is exactly the instance $d_{2,1} = 2$.

Next separate a bounded residue-dependent base from an unbounded unary tail. For $0 \le s < p$ and $0 \le t < q$, define

$$A_s = C(s), \qquad B_t = C(-t), \qquad A_0 = B_0 = \varepsilon.$$

For positive $r = pm + s$ and negative $r = -(qm + t)$,

$$\boxed{C(pm + s) = A_s 0^m,} \qquad \boxed{C(-(qm + t)) = B_t 1^m.}$$

Definition 5.3 (Completion frontier)**.** Define

$$K_{p,q} = \max\left(\max_{0 \le s < p} |A_s|, \max_{0 \le t < q} |B_t|\right).$$

Theorem 5.4 (Exact almost-cycle frontier)**.** *For every coprime* $(p, q) \neq (1, 1)$*,*

$$\boxed{K_{p,q} = p + q - 1.}$$

For $(p, q) = (1, 1)$*,* $K_{1,1} = 0$*.*

Proof. The modular bounds in theorem 3.3 give the upper bound $K_{p,q} \le p + q - 1$. If $p > q$, the base residual q has

$$A_q = C(q) = 1^{p-1}0^q,$$

whose length is $p + q - 1$. If $q > p$, the symmetric base is

$$B_p = C(-p) = 0^{q-1}1^p.$$

The remaining equal-weight coprime case is $(1, 1)$. □

Combining theorems 4.2 and 5.4 gives

$$\boxed{K_{p,q} = g_{p,q} - 1}$$

for every nontrivial pair. The extremal completion is literally one symbol short of a fundamental balanced cycle, and the same word witnesses the lower bound in theorem 5.2. This is the structural link between the original bounded-search observation and the general finite frontier.

6 Lossless frontier-to-control factorization

A classical automata technique can keep a bounded stack prefix in finite control; modern discussions explicitly trace deterministic top-k variants to classical constructions [13]. Our use of that idea is representation-specific: the preceding section identifies *which* frontier is relevant, proves it is bounded, gives its exact worst-case size, and equips it with shortest-completion semantics.

Map the canonical completion representations to

$$A_s 0^m \longmapsto (P_s, 0^m Z_0), \qquad B_t 1^m \longmapsto (N_t, 1^m Z_0),$$

with the explicit convention

$$P_0 = N_0 = B.$$

Thus the zero-residue base $A_0 = B_0 = \varepsilon$ is represented by the common state B rather than by two undeclared states. Use the core state set

$$Q_{\mathrm{core}} = \{B\} \cup \{P_s : 1 \le s < p\} \cup \{N_t : 1 \le t < q\}.$$

Then

$$|Q_{\mathrm{core}}| = p + q - 1.$$

Give states the potentials

$$\Phi(B) = 0, \qquad \Phi(P_s) = s, \qquad \Phi(N_t) = -t,$$

and the stack the weight

$$W = pN_0 - qN_1.$$

Theorem 6.1 (Lossless completion-frontier factorization)**.** *The map above is injective on normalized representations and has an explicit inverse. For every processed prefix w, if $\sigma \in Q_{\mathrm{core}}$ is the current control state and S is the stack, the normalized configuration satisfies*

$$\boxed{\rho_{p,q}(w) = W(S) + \Phi(\sigma).}$$

It therefore determines exactly the residual, the prefix quotient, and the canonical shortest completion $C(\rho_{p,q}(w))$.

Proof. The state records the bounded residue-dependent base A_s or B_t, including the sign and residue, while the unary stack tail records m. This data reconstructs the canonical word uniquely. The potential identity is just the Euclidean decomposition $r = pm + s$ or $r = -(qm + t)$ expressed in state-plus-stack form. Theorems 3.3 and 3.6 then give quotient and witness recovery. □

Running example ($p = 3, q = 2, r = 4$). The earlier shortest completion $C(4) = 100$ decomposes as

$$4 = 3 \cdot 1 + 1, \qquad A_1 = C(1) = 10, \qquad C(4) = A_1 0.$$

The direct factorization therefore stores the same object as

$$(P_1, 0Z_0).$$

Indeed, $W(0Z_0) = 3$ and $\Phi(P_1) = 1$, so $W + \Phi = 4$. The example makes explicit that the finite-control base and the unary stack tail are not new information: they are a lossless coordinate change for the canonical completion word.

For the lower-bound statement only, fix the following representation class. A *direct unary-tail representation* uses exactly the physical stack $0^m Z_0$ on the positive side and $1^m Z_0$ on the negative side, with no additional boundary marker carrying residue or sign information. Consequently every boundary residual

$$0, 1, \ldots, p-1, -1, \ldots, -(q-1)$$

has the same exact physical stack Z_0 (that is, $m = 0$), and all information distinguishing those boundary residuals is carried by finite control. The proposition below is only about this explicitly fixed class.

Within this direct unary-tail representation class the state count is exact.

Proposition 6.2 (Representation-relative normalized-state complexity)**.** *Any lossless direct unary-tail representation in which the boundary residuals*

$$0, 1, \ldots, p-1, -1, \ldots, -(q-1)$$

all have the same exact stack Z_0 needs at least $p+q-1$ normalized states. The core above attains the bound.

Proof. There are $p+q-1$ distinct residuals, hence distinct prefix quotients by theorem 3.6. With the identical physical stack Z_0 and no additional boundary symbol available in this representation class, the finite control must distinguish them. □

Remark 6.3. The scope of theorem 6.2 is essential. It is not an unrestricted DPDA lower bound; section 7 deliberately changes the stack representation and uses fewer states.

7 A compact four-symbol realization

We now pair one positive and one negative residue role in the same control state, separating them by the stack top. This is a secondary descriptional construction rather than the conceptual starting point.

Let

$$m = \max(p, q), \qquad \Gamma_4 = \{0, 1, X, Z_0\}.$$

Define

$$a_s = \begin{cases} 0, & s = 0, \\ s - p, & 1 \le s < p, \end{cases} \qquad b_t = \begin{cases} 0, & t = 0, \\ q - t, & 1 \le t < q. \end{cases}$$

For $r > 0$, with $s = r \bmod p$, there is a unique $k \ge 1$ such that

$$r = pk + a_s,$$

and we encode

$$\eta(r) = (R_s, 0^k Z_0).$$

For $r < 0$, with $t = (-r) \bmod q$, there is a unique $k \ge 1$ such that

$$r = -qk + b_t,$$

and we encode

$$\eta(r) = (R_t, 1^k X).$$

For the no-endmarker realization define additionally

$$\eta_0(0) = (Z, Z_0),$$

and for the endmarker realization define

$$\eta_{\dashv}(0) = (R_0, Z_0).$$

For $r \neq 0$, both $\eta_0(r)$ and $\eta_\dashv(r)$ use the nonzero encoding $\eta(r)$ displayed above. The marker X is the negative-side sentinel, whereas Z_0 is the positive/zero sentinel. Thus a state R_i can host a positive residue class on top 0 and a negative residue class on top 1.

Theorem 7.1 (Compact paired-sign realization)**.** *For every coprime $p, q \geq 1$ there is an ordinary final-state DPDA without a right endmarker with*

$$\boxed{|Q| = \max(p, q) + 1, \qquad |\Gamma| = 4.}$$

With a mandatory right endmarker there is a DPDA with

$$\boxed{|Q| = \max(p, q), \qquad |\Gamma| = 4.}$$

Both constructions admit an equivalent push-at-most-two form with the same state and stack-alphabet counts.

Proof. Without an endmarker, use states $Z, R_0, \ldots, R_{m-1}$, with Z the unique final zero state. Positive configurations use top 0 and negative configurations use top 1; Z_0 and X distinguish the two near-zero boundaries. The complete stable, boundary, zero, and endmarker transition schemas are given in section A. Each rule is obtained by applying the residual updates $r \mapsto r + q$ and $r \mapsto r - p$ to the explicit encoding, and the identities there verify the resulting target configuration exactly.

With an endmarker, omit Z and represent zero by (R_0, Z_0). The endmarker transition is defined only on this state/top pair; after consuming the mandatory $\dashv$, R_0 is the accepting final state. Hence nonzero exact multiples that also use control state R_0 but have top 0 or 1 cannot accept.

For push-at-most-two normalization, only the heavier direction can require a long unary push. If $p > q$, exactly $p - q$ suffix roles are needed, all with top 1, and the stable paired-sign representation leaves exactly the $p - q$ cells $(R_i, 1)$ for $q \leq i < p$ unused. Overlay the suffix roles onto those cells. If $q > p$, the symmetric unused top-0 cells are used. A cross-sign first step is bounded directly by $Z_0 \mapsto 1X$ or $X \mapsto 0Z_0$. No new state or stack symbol is required. □

The running example in the compact encoding. For $p = 3, q = 2, r = 4$, we have $s = 1$ and $a_1 = 1 - 3 = -2$. Hence

$$4 = 3 \cdot 2 - 2, \qquad \eta(4) = (R_1, 00Z_0).$$

Compare this with the direct-factorization configuration $(P_1, 0Z_0)$ above: the two representations use different coordinates, but both decode to the same residual 4 and therefore to the same quotient and canonical shortest completion 100.

For the explicit push-at-most-two schema, the number of transient suffix roles is $|p-q|$, and the total number of transition rules is

$$\boxed{|\delta| = 3(p + q) + 2 + |p - q|.}$$

The displayed state counts arise from the specific paired-role layout used here: each R_i can host at most the positive role assigned to its top-0 cell and the negative role assigned to its top-1 cell, while the no-endmarker convention uses the separate zero/final state Z. With a mandatory endmarker, zero shares (R_0, Z_0). We make no unrestricted DPDA state-optimality claim, nor a lower-bound claim outside this explicitly defined layout.

8 Configuration semantics and witness decoding

The central semantic point is not the state count but the information represented by a normalized configuration.

Theorem 8.1 (Configuration–quotient–decoder theorem)**.** *For either normalized semantic construction, let $\eta(r)$ denote the unique stable configuration representing residual r. Then*

$$\eta(r) \leftrightarrow r \leftrightarrow \{z : px - qy = r\}$$

and

$$\eta(r) \xrightarrow{1} \eta(r+q), \qquad \eta(r) \xrightarrow{0} \eta(r-p).$$

Consequently a normalized configuration determines simultaneously:

(i) the exact residual r;

(ii) the exact prefix left quotient;

(iii) the exact directed distance to acceptance $\ell_{\min}(r)$;

(iv) the canonical shortest continuation $C(r)$.

Proof. The encoding is injective and explicitly decodable by construction. The transition identities update the represented residual by $+q$ on input 1 and $-p$ on input 0. Quotient equivalence is theorem 3.6; distance and witness are theorems 3.3 and 4.1. □

The word "decoder" is semantic: the DPDA is not claimed to be an output transducer. Rather, its stable configuration is a certificate from which the canonical shortest continuation can be recovered by a fixed mathematical map.

9 Executable validation

The theorems above are proved symbolically. Independent scripts were used only as adversarial consistency checks.

For the original 2:1 machine, every binary word through length 18—524,287 inputs—was checked for the residual invariant, reachable-store normal form, completion normalization, independent minimum completion length, and depth-two support visibility. For the general arithmetic, all coprime pairs $1 \le p, q \le 60$ and lengths through 150 were checked against independent shortest-distance oracles, including 25,281,628 distance checks. The four-symbol push-at-most-two construction was tested for every coprime pair $1 \le p, q \le 45$, both acceptance conventions, and every binary word through length 8, for 1,282,610 complete machine runs. A separate formula audit through $p, q \le 80$ verified the suffix-role and transition-count formulas for 3,931 coprime parameter pairs. All checks passed.

These experiments are evidence against transcription and boundary-case errors; they are not proof premises. The complete reference validation suite is distributed as `paper_validation_suite.py` in the source-design repository `https://github.com/alperenbutun/dpda-ones-double-zeros`. It uses only the Python standard library; running `python paper_validation_suite.py --paper` executes the parameter ranges

reported above and emits a machine-readable JSON summary. The second source-design repository used for figure 2 is linked directly in its caption, and related implementation material can also be reached from the author's GitHub profile, `https://github.com/alperenbutun`.

10 Related work and novelty boundary

Deterministic one-counter automata have a long history [1, 2]. Signed count differences and residue components in finite control are explicit in modern counter-automata constructions [3]. Group/counter-register automata provide a still broader register perspective [4]. Therefore the recognizability of $L_{p,q}$ by deterministic counter-like memory is background rather than a contribution.

Store languages and intermediate pushdown configurations are an established object of study [6]; recent work also develops further characterizations and complexity results for language acceptors with pushdown stores and counter extensions [12]. Likewise, shortest paths in one-counter systems have been investigated in generality [5]; regular geodesic normal forms are a classical topic in group-theoretic settings [7]; and descriptional-complexity work explicitly studies counting with finite machines and state/pushdown-alphabet resources [10, 8, 9]. Unary DPDAs can themselves be exponentially succinct relative to finite-state descriptions [11], reinforcing the need to state representation-dependent resource bounds carefully. Finally, deterministic conversions that keep a bounded top portion of the stack in finite control are classical and are discussed explicitly in recent WPDA work [13].

These results narrow the novelty claim of the present paper. We do not propose a new one-counter model, a new general top-k simulation, or a new theory of geodesic languages. The contribution is the exact family-specific chain rooted in the original 2023 completion-aware DPDA: its state-dependent reachable-store normal form; shortest-completion interpretation; tight original-machine and general visibility bounds; exact almost-cycle frontier; lossless completion-semantic factorization; and the simultaneous quotient/distance/shortest-witness interpretation of the resulting normalized configuration. The compact four-symbol automata are useful explicit realization theorems, but they are not presented as the invention of residue compression.

11 Discussion

The original machine is valuable precisely because it is more complicated than the obvious counter solution. Its side states, cancellations, and post-input completion rules expose a latent representation of "what is still needed." Once the reachable-store family is characterized, the apparently global stack search collapses to a bounded observation. Generalizing that observation leads to the canonical two-block completions, whose geometry explains both the exact visibility depth and the exact finite frontier.

The two memory parameters should not be conflated. The visibility depth

$$d_{p,q} = \max(p, q)$$

asks how far into a canonical completion one must inspect to know which symbol types occur. The frontier size

$$K_{p,q} = p + q - 1$$

measures the largest complete residue-dependent base before the unary tail begins. Their common extremal word makes them related, but they quantify different questions.

The state counts are likewise representation-dependent. The direct three-symbol factorization is semantically transparent and has $p + q - 1$ normalized states. Adding the boundary marker X allows positive and negative residue classes to share control states, reducing the count to about half when p and q are comparable. This is a concrete example of the broader fact that state and stack-alphabet resources can trade against one another. It is also why unrestricted state-minimality should not be inferred from a clean semantic factorization.

Several problems remain open. A meaningful unrestricted lower bound would require a precisely fixed descriptional model, including stack alphabet, replacement length, and acceptance convention. Transition-minimality of the explicit bounded-push schema is also open beyond the stated representation class. Further work could test the completion-frontier and decoder package against still broader subclasses of counter and pushdown systems; the present claims remain deliberately family-specific and representation-aware.

12 Conclusion

A completion-aware DPDA for $L_{2,1}$ led to a structure that is not visible from recognition alone. Its reachable stores admit an exact normal form; a state potential converts stack behavior into an arithmetic residual; its end-of-input rules extract a shortest continuation; and the design's bounded-search intuition becomes the tight theorem $d_{2,1} = 2$.

For the full family $L_{p,q}$, residuals are exactly prefix quotients and shortest completions are directed geodesics. The fundamental balanced cycle has length $p + q$; maximal cycle deletion gives the unique minimum Parikh pair; canonical shortest completions have exact visibility depth $\max(p, q)$ and exact finite frontier $p + q - 1$. Factoring that frontier into control preserves quotient and shortest-witness semantics, while a paired-sign store encoding yields compact four-symbol DPDAs. The resulting picture connects an operational stack design to quotient geometry, bounded memory, and exact continuation semantics without treating the classical counter substrate as the novelty.

Future work. Two directions are especially natural. First, the explicit compact constructions invite lower bounds under a fixed descriptional model that simultaneously constrains states, stack alphabet, replacement length, and acceptance convention. Second, the completion-frontier viewpoint may extend to higher-dimensional commutative counting constraints, where the kernel is no longer generated by a single balanced cycle and shortest-completion geometry becomes genuinely multidimensional.

A Symbolic transition schema for the compact DPDA

This appendix records the unbounded-fixed-push semantic transition identities. The bounded-push compiler preserves them by replacing long homogeneous writes with forced

suffix chains.

A.1 No-endmarker model

Use states

$$Q = \{Z\} \cup \{R_i : 0 \le i < m\}, \qquad m = \max(p, q),$$

with Z initial and final and initial stack Z_0. Extend the encoding from section 7 by

$$\eta_0(0) = (Z, Z_0),$$

and let $\eta_0(r) = \eta(r)$ for $r \neq 0$. When $\eta_0(r) = (\sigma_r, S_r)$, an equation of the form $\delta(\cdot) = \eta_0(r)$ means that the current boundary sentinel (which is the whole stack at that moment) is replaced by S_r and control enters σ_r.

For $0 \le s < p$, define

$$s' = (s + q) \bmod p, \qquad c_s = \frac{a_s + q - a_{s'}}{p}.$$

The positive stable rules are

$$\delta(R_s, 0, 0) = (R_s, \varepsilon), \qquad \delta(R_s, 1, 0) = (R_{s'}, 0^{c_s+1}).$$

If the first rule pops the last 0, the sentinel Z_0 is exposed and the residual has become a_s; the corresponding boundary rule is therefore

$$\boxed{\delta(R_s, \varepsilon, Z_0) = \eta_0(a_s)} \qquad (0 \le s < p).$$

For $s = 0$ this goes to the zero configuration; for $s > 0$ it performs the unique cross-sign conversion to the negative encoding.

For $0 \le t < q$, define

$$t' = (t + p) \bmod q, \qquad d_t = \frac{p - b_t + b_{t'}}{q}.$$

The negative stable rules are

$$\delta(R_t, 1, 1) = (R_t, \varepsilon), \qquad \delta(R_t, 0, 1) = (R_{t'}, 1^{d_t+1}).$$

If the first rule pops the last 1, the sentinel X is exposed and the residual has become b_t; hence

$$\boxed{\delta(R_t, \varepsilon, X) = \eta_0(b_t)} \qquad (0 \le t < q).$$

Finally the two input moves from exact zero are

$$\boxed{\delta(Z, 1, Z_0) = \eta_0(q), \qquad \delta(Z, 0, Z_0) = \eta_0(-p).}$$

No binary-input transition is defined on the transient boundary pairs (R_s, Z_0) or (R_t, X), so the displayed ε-moves do not compete with input-consuming moves. Thus the schema is deterministic.

Correctness follows directly from

$$\begin{aligned} pk + a_s - p &= p(k-1) + a_s, \\ pk + a_s + q &= p(k + c_s) + a_{s'}, \\ -qk + b_t + q &= -q(k-1) + b_t, \\ -qk + b_t - p &= -q(k + d_t) + b_{t'}. \end{aligned}$$

The boundary equations simply evaluate the same identities at $k = 1$, and the zero rules apply the residual updates $0 \mapsto q$ and $0 \mapsto -p$.

A.2 Right-endmarker model

Use only $R_0, \dots, R_{m-1}$ and define

$$\eta_{\dashv}(0) = (R_0, Z_0), \qquad \eta_{\dashv}(r) = \eta(r) \quad (r \neq 0).$$

The stable rules above are unchanged. The positive boundary rule is required only for $1 \leq s < p$,

$$\boxed{\delta(R_s, \varepsilon, Z_0) = \eta_{\dashv}(a_s)},$$

because when $s = 0$ the last-0 pop already leaves the stable zero configuration (R_0, Z_0). The negative boundary rules remain

$$\boxed{\delta(R_t, \varepsilon, X) = \eta_{\dashv}(b_t)} \qquad (0 \leq t < q).$$

At zero define the two binary moves and the accepting endmarker move by

$$\boxed{\delta(R_0, 1, Z_0) = \eta_{\dashv}(q), \qquad \delta(R_0, 0, Z_0) = \eta_{\dashv}(-p),}$$

$$\boxed{\delta(R_0, \dashv, Z_0) = (R_0, Z_0).}$$

There is no ε-move on (R_0, Z_0) and no endmarker move on top 0, top 1, or sentinel X. Hence zero can continue reading binary input or consume $\dashv$, while every nonzero stable configuration rejects at the endmarker. The determinism condition is again immediate from the disjoint stable and transient state/top roles.

A.3 Push-at-most-two compilation

Assume $p > q$; the case $q > p$ is obtained by exchanging 0 and 1 and reversing the sign roles. In this regime the only replacements that can exceed length two are writes of homogeneous blocks of 1s. For $u \in \{0, \dots, q-1\}$ let

$$u' = (u + p) \bmod q, \qquad d_u = \frac{p - b_u + b_{u'}}{q}.$$

Because addition by p permutes the residues modulo q, each target $v \in \{0, \dots, q-1\}$ has a unique predecessor

$$u_v = (v - p) \bmod q.$$

Create abstract suffix roles

$$H_{v,j}, \qquad 1 \leq j \leq d_{u_v} - 1,$$

with forced rules

$$\delta(H_{v,1}, \varepsilon, 1) = (R_v, 11),$$

and for $j > 1$,

$$\delta(H_{v,j}, \varepsilon, 1) = (H_{v,j-1}, 11).$$

Thus $H_{v,j}$ means that exactly j further one-symbol expansions remain before the stable target R_v is reached.

A long stable rule

$$\delta(R_u, 0, 1) = (R_{u'}, 1^{d_u+1})$$

is left unchanged when $d_u \le 1$; when $d_u \ge 2$ it is compiled as

$$\boxed{\delta(R_u, 0, 1) = (H_{u',d_u-1}, 11),}$$

followed by the forced suffix chain above.

Lemma A.1 (Cross-sign suffix availability)**.** *Assume $p > q$. For $0 \le s < p$, put $n = p - s$, let $v = n \bmod q$, and let $u_v = (v - p) \bmod q$ be the unique predecessor of v under addition by p modulo q. If*

$$h = \left\lceil \frac{p - s}{q} \right\rceil,$$

then

$$\boxed{h \le d_{u_v}.}$$

More precisely, $d_{u_v} - h = \lfloor s/q \rfloor$.

Proof. Since $v \equiv p - s \pmod q$, one has $u_v \equiv -s \pmod q$. By the definition of b_t, this gives $b_{u_v} = s \bmod q$ (with residue 0 represented by $b_0 = 0$). Also $qh = p - s + b_v$. Therefore

$$q(d_{u_v} - h) = p - b_{u_v} + b_v - (p - s + b_v) = s - b_{u_v} = q \left\lfloor \frac{s}{q} \right\rfloor.$$

The claimed inequality follows. The endpoint $s = 0$ is the zero-to-negative move and gives equality $h = d_{u_v}$. □

Lemma A.2 (Exact suffix-role count)**.** *For $p > q$, every $d_u \ge 1$, and*

$$\sum_{u=0}^{q-1} d_u = p, \qquad \sum_{u=0}^{q-1} (d_u - 1) = p - q.$$

Proof. Because $0 \le b_u, b_{u'} \le q - 1$ and $p > q$, the positive integer $p - b_u + b_{u'}$ is at least $p - q + 1$; since its quotient by q is the integer d_u, one has $d_u \ge 1$. The map $u \mapsto u' = (u + p) \bmod q$ is a permutation because $\gcd(p, q) = 1$. Hence

$$\sum_{u=0}^{q-1} d_u = \frac{1}{q} \left(qp - \sum_{u=0}^{q-1} b_u + \sum_{u=0}^{q-1} b_{u'} \right) = p.$$

Subtracting q gives the second identity. □

A cross-sign or zero rule whose semantic target is

$$(R_v, 1^h X), \qquad h \ge 1,$$

is direct when $h = 1$, and for $h \ge 2$ is compiled as

$$\boxed{\delta(\cdot, \cdot, Z_0) = (H_{v,h-1}, 1X),}$$

again followed by the same suffix chain. For the positive-to-negative boundary residual $a_s = -(p - s)$, the required exponent is $h = \lceil (p - s)/q \rceil$. By theorem A.1, the required suffix role is already among those created; the zero-to-negative move $0 \mapsto -p$ is the endpoint $s = 0$ of the same lemma. All writes of 0s have length at most two when $p > q$:

directly from the definition of a_s, the integer $c_s = (a_s + q - a_{s'})/p$ is 0 or 1. Therefore those replacements have length $c_s + 1 \le 2$ and need no compiler states.

By theorem A.2, the number of suffix roles is exactly

$$\sum_{u=0}^{q-1}(d_u - 1) = p - q.$$

The stable paired-sign semantics leave exactly the $p - q$ state/top-1 cells

$$(R_i, 1), \qquad q \le i < p,$$

unused. Choose any bijection between these cells and the suffix roles $H_{v,j}$ and interpret the corresponding state/top pair as that role. Because those cells have no stable meaning, the forced ε-moves introduce no transition conflict. This realizes every semantic replacement with length at most two without increasing either the state set or the four-symbol stack alphabet. For $q > p$, the symmetric construction uses the unused top-0 cells.

The semantic schema contains $2p + 2q$ stable rules, $p + q$ boundary rules, and two zero-input rules in the no-endmarker model, for $3(p+q)+2$ rules before compilation. The endmarker model replaces one positive boundary rule by the endmarker rule and has the same count. The $|p - q|$ forced suffix rules therefore give

$$\boxed{|\delta| = 3(p + q) + 2 + |p - q|.}$$

B Summary of results and scope

Result	Status	Scope
Original-machine reachable-store normal form	proved	for the formalized 2023 design
Original-machine completion extraction	proved	completion is endmarker/post-input behavior
Original-machine depth two	proved, tight	symbol-presence query only
$d_{p,q} = \max(p, q)$	proved, tight	canonical shortest-completion representation
$K_{p,q} = p + q - 1$	proved, tight	residue-unary completion frontier
Direct-factorization $p + q - 1$ lower bound	proved	representation-relative only
Four-symbol compact DPDA	proved	explicit construction; no unrestricted state-optimality claim
Push-at-most-two with same state count	proved	explicit compiler / four-symbol store
Global DPDA state optimality	open	not claimed
Generic one-counter recognition	prior art	background
Generic bounded stack prefix in control	prior art	background
Regular geodesic normal forms	prior art	background

References

[1] L. G. Valiant and M. S. Paterson. Deterministic one-counter automata. *Journal of Computer and System Sciences*, 10(3):340–350, 1975.

[2] S. Böhm, S. Göller, and P. Jančar. Equivalence of deterministic one-counter automata is NL-complete. In *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC 2013)*, pp. 131–140, 2013. doi:10.1145/2488608.2488626.

[3] M. Kutrib and A. Malcher. Reversible computations of one-way counter automata. In *Proceedings of NCMA 2022*, EPTCS 367, pp. 126–142, 2022. arXiv:2208.14720.

[4] M. Elder, M. Kambites, and G. Ostheimer. On groups and counter automata. *International Journal of Algebra and Computation*, 18(8):1345–1364, 2008. arXiv:math/0611188.

[5] D. Chistikov, W. Czerwiński, P. Hofman, M. Pilipczuk, and M. Wehar. Shortest paths in one-counter systems. *Logical Methods in Computer Science*, 15(1):19:1–19:28, 2019. arXiv:1510.05460.

[6] O. H. Ibarra and I. McQuillan. On store languages of language acceptors. *Theoretical Computer Science*, 745:114–132, 2018. arXiv:1702.07388.

[7] W. D. Neumann and M. Shapiro. Regular geodesic normal forms in virtually abelian groups. *Bulletin of the Australian Mathematical Society*, 55(3):517–519, 1997. arXiv:math/9702203.

[8] T. Masopust. A note on limited pushdown alphabets in stateless deterministic pushdown automata. arXiv:1208.5002, 2012.

[9] G. Pighizzini. Deterministic pushdown automata and unary languages. *International Journal of Foundations of Computer Science*, 20(4):629–645, 2009. arXiv:0905.1248.

[10] D. Chistikov. Notes on counting with finite machines. In *34th International Conference on Foundation of Software Technology and Theoretical Computer Science (FSTTCS 2014)*, LIPIcs 29, pp. 339–350, 2014. doi:10.4230/LIPIcs.FSTTCS.2014.339.

[11] D. Chistikov and R. Majumdar. Unary pushdown automata and straight-line programs. In *Automata, Languages, and Programming: 41st International Colloquium (ICALP 2014), Part II*, LNCS 8573, pp. 146–157. Springer, 2014. doi:10.1007/978-3-662-43951-7_13.

[12] O. H. Ibarra and I. McQuillan. Language acceptors with a pushdown: characterizations and complexity. *International Journal of Foundations of Computer Science*, 36(3):345–370, 2025. doi:10.1142/S0129054124430044.

[13] A. Butoi, B. DuSell, T. Vieira, R. Cotterell, and D. Chiang. Algorithms for weighted pushdown automata. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 9669–9680. Association for Computational Linguistics, 2022. doi:10.18653/v1/2022.emnlp-main.656.